Reference Architecture Brief: Composable APIs, Services and Applications

19 March 2025 - ID G00820726 - 18 min read

By: Brad Dayley

Initiatives:Software Architecture and Integration for Technical Professionals; Adopt Modern Architectures and Technologies; Build and Deliver Software to Drive Business Results

> Building composable applications requires an architecture that enables agility, flexibility, integration and modularity. A composable application architecture of apps, APIs and services provides software architects and engineers with the optimal approach to meet those needs.

Architecture Brief

Composable architecture accelerates transformation, reduces costs and improves UX by enabling rapid adaptation to new user requirements and technology advancements. Additionally, as requirements evolve, architects must be able to add new architecture capabilities while continuing to leverage existing data and applications. A composable architecture comprising modern multiexperience apps, APIs and decoupled services provides the agility and flexibility to satisfy the needs of the digital business.

This Reference Architecture Brief describes a composable architecture that allows software architects and software engineers to create a consistent architecture across a heterogeneous and distributed set of systems, services and applications. Teams can use this architecture to incrementally modernize monolithic, tightly coupled application stacks that are ill-equipped to support innovation. Composability enables teams to iteratively evolve an application's architecture as the requirements and implementation constraints become clear.

Architecture Use Cases

This architecture is optimal for:

- **Agile applications:** Creating applications with a high degree of agility and flexibility that enables rapid support for changing business needs.

This research note is restricted to the personal use of .

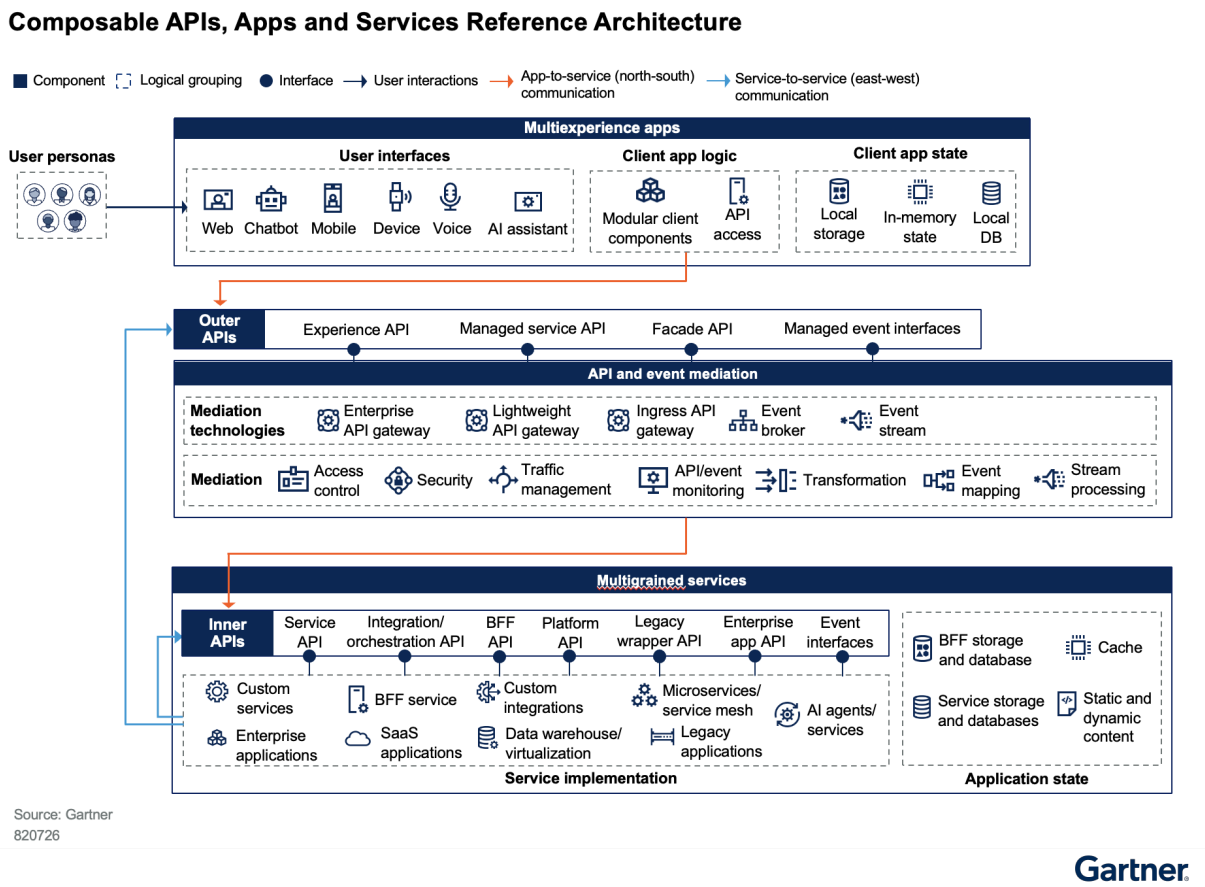
- **Composable applications:** Creating applications through composition of functionality, logic and data from multiple systems, services and applications.
- **AI-enabled applications:** Creating applications with AI capabilities, such as AI agents, as well as integrating AI capabilities into distributed application architecture.
- **Multiexperience applications:** Creating applications with multiple fit-for-purpose apps that align with individual roles and workflows.
- **Application modernization:** Modernizing legacy applications by updating, replacing and refactoring components independent of each other, at a rate the organization can tolerate.
- **Enterprise application integration:** Enabling integration between enterprise applications.

Architecture Diagram

The composable application architecture shown in Figure 1 includes multiexperience apps, multigrained services, and mediated APIs and events. These components give applications consistent access to data and functionality in back-end systems and services.

This research note is restricted to the personal use of .

Figure 1: Composable Apps, APIs and Services Reference Architecture



▶ Download an Editable Version of This Figure

A key principle of this architecture is to decouple the user experiences from the back-end service implementations through well-defined APIs and event interfaces between the back-end services. Such decoupling enables each component of the architecture to be built, tested and deployed independent of the others.

Instead of a single, complex application, the user experience comprises multiple purpose-built apps that align with the specific workflows required by different personas working through various interfaces. The architecture then allows functionality and data for the apps to be distributed across a variety of interconnected and reusable back-end systems and services.

This research note is restricted to the personal use of .

Each of these components of the architecture is critical to the full architecture implementation and evolution. Loose coupling allows you to break down complex modernization and implementation into reasonable chunks. For more detailed guidance on implementing this type of architecture, see MASA: Create Agile Application Architecture With Composable Apps, APIs and Services.

Architecture Capabilities & Components

To implement this architectural approach, you will need to build or enable the following components:

- Multiexperience apps
- Outer APIs
- API and event mediation
- Inner APIs
- Multigrained services

Multiexperience Apps

These apps are user interfaces that provide fit-for-purpose workflows. They focus on the workflows that particular personas follow as they perform their specialized tasks. This approach provides the opportunity to build smaller, narrowly focused apps that can:

- Use multiple services from the back end
- Deploy on a variety of channels and modalities, such as web, mobile, desktop, voice, chatbots and AI assistants

Example Technologies

The following are examples of the types of technologies that may be used in creating multiexperience apps:

- **Web apps:**
  - Angular, Microsoft Blazor, React, Vue.js

This research note is restricted to the personal use of .

- **Mobile apps:**
  - Flutter, Kotlin, Microsoft .NET Multi-platform App UI (MAUI), React Native, Swift
- **Low-code application platforms:**
  - Mendix, Microsoft Power Apps, OutSystems

Key Characteristics

Multiexperience apps have the following characteristics:

- Employ a modular structure that allows individual components to be easily adapted or reused in other apps.
- Are decoupled from the back-end implementation via APIs and events to minimize dependencies on back-end implementation details.
- Separate app logic from UI logic to reduce dependencies.
- Support multiexperience workflows that can traverse multiple devices and modalities.
- Are fit-for-purpose and aligned with specific workflows and user personas.
- Provide a consistent look and feel when a user navigates between them.
- Support single sign-on to allow users to navigate between apps.
- Enable users to pick up where they left off if they need to leave the app in the middle of a workflow.
- Enable users to start a workflow on one device and finish it on a separate device.

Related Architecture Research

MASA: Create Agile Application Architecture With Composable Apps, APIs and Services

Using MASA to Deliver Agile Enterprise Application Architecture

Reference Architecture Brief: Agile Web Application Architecture

JavaScript: Top Use Cases, Frameworks and Architecture Constraints

This research note is restricted to the personal use of .

Outer APIs

Outer APIs are tailored specifically to the needs and requirements of multiexperience apps, integrations, automation and other use cases. Exposed via the mediation layer, outer APIs abstract the complexities and implementation details of underlying systems and services away from the consuming developers.

This abstraction supports agility, performance, scalability, security and resilience. It also avoids unnecessary dependencies on back-end implementations. Additionally, outer APIs act as a contract between the teams creating user experiences and the ones delivering back-end functionality, essentially allowing them to operate independently in terms of technology choices and delivery cadences.

Example Technologies

The following are examples of the types of technologies that may be used to define and simplify management of outer APIs:

- **API formats:**
  - AsyncAPI, GraphQL, gRPC (Protocol Buffers), OpenAPI Specification (REST APIs)

Key Characteristics

Outer APIs have the following characteristics:

- Are defined using well-known standard specifications such as OpenAPI.
- Support authentication and authorization.
- Support load balancing and traffic control.
- Are well-documented and discoverable by developers.
- Allow interface connectivity to include both event communication and API communication.
- Are designed using a consumer-centric approach that aligns the functionality provided by the API with the needs of the consuming app.

This research note is restricted to the personal use of .

- Provide a consistent layer for one or more apps or integrations to access back-end data, resources and functionality.
- Are treated as part of the developer platform by consuming developers.
- May serve multiple apps or may be unique to a specific app.
- Have a roadmap and life cycle strategy that includes how and when the API version may change.
- Abstract the implementation details of underlying systems, services and applications away from the consuming developers.

Related Architecture Research

How to Design Great APIs

How to Deliver Sustainable APIs

Choosing an API Format: REST Using OAS, GraphQL, gRPC or AsyncAPI

Reference Architecture Brief: Event-Driven Application Architecture

API and Event Mediation

API and event mediation utilizes technologies to abstract the implementation details behind the provider's API and event interfaces away from consumers. It allows providers of the APIs and events to monitor the interfaces, route traffic, and apply security and access policies. Mediation capabilities include translating protocols, data formats and structures, as well as routing and orchestrating calls across service implementations using APIs and event channels within the architecture.

The two key benefits of using mediation for APIs and events are:

- Mediation minimizes dependencies and direct coupling between API consumers and underlying systems. For example, mediation decouples multiexperience apps from the implementation complexities and constraints of the systems, services and applications that they rely on.
- Mediation provides consistency in the way that security, access and governance policies are defined and applied, making distributed architecture easier to scale.

This research note is restricted to the personal use of .

Example Technologies

The following are examples of the types of technologies that may be used in API and event mediation:

- **API formats:**
  - AsyncAPI, GraphQL, gRPC (Protocol Buffers), OpenAPI Specification (REST APIs)
- **API gateways:**
  - Akamai API Gateway, Amazon API Gateway, Apache APISIX, Axway Amplify, Google Cloud API Gateway, Google Cloud Apigee, Gravitee.io, IBM API Connect, Kong Konnect, KrakenD, Microsoft Azure API Management, Salesforce (MuleSoft) Universal API Management, Software AG webMethods API Gateway, Solo.io Gloo Gateway, Spring Cloud Gateway, TIBCO Cloud API Management, Tyk Gateway, WSO2 API Manager
- **GraphQL technologies:**
  - Apollo GraphOS, AWS AppSync, Azure API Management, Solo.io Gloo Edge, Tyk Universal Data Graph
- **Event technologies:**
  - Amazon Kinesis, Apache Kafka, Azure Event Hubs, Azure Service Bus, Google Cloud Pub/Sub, IBM MQ, RabbitMQ, Solace PubSub+ Platform

Key Characteristics

API and event mediation has the following characteristics:

- Allows connectivity to include both event communication and API communication.
- Enforces policies for managing security and traffic.
- Exposes outer APIs for the APIs that it manages.
- Routes traffic to provide connectivity to back-end systems and services.

This research note is restricted to the personal use of .

- Allows policies, such as security, traffic management, monitoring and transformation policies, to be defined declaratively and applied to one, many or all APIs.
- Allows policy updates to be made once and automatically applied to all associated APIs.
- Is deployed wherever APIs are deployed.
- Enables API versioning and custom APIs.
- Automates throttling and management of inbound and outbound traffic.
- Collects essential information/metrics that can be used to evaluate the effectiveness of the API, identify shortcomings or highlight aspects that require improvement.
- Uses the appropriate API gateway technology for the type of traffic being mediated.
- Simplifies security and traffic management by removing the burden of implementing them from each API implementation team.
- Increases consistency and mitigates the risk of human error when organizations are implementing security and access control policies.
- Does not include heavy format/payload modifications or business, integration and orchestration logic.
- Insulates the interfaces used by client apps from the disparity and constraints of back-end services.
- Provides consistency to access management, traffic flow and monitoring of the interfaces.

Related Architecture Research

Decision Point for Selecting the Right API Mediation Technology

Reference Architecture Brief: API Management

Choosing Event Brokers to Support Event-Driven Architecture

When to Use a Service Mesh in Cloud-Native Architectures

This research note is restricted to the personal use of .

How to Deliver Sustainable APIs

How to Evaluate API Management Solutions

Inner APIs

Inner APIs enable access to multigrained services providing business logic, data, resources and other back-end functionality required by other apps, services and systems in a distributed architecture. They are generally tailored to the domain constraints of the service to support access from multiple mediators.

The purpose of inner APIs is to provide general access to domain functionality and resources while still abstracting the complexities and implementation details of underlying systems and services. This abstraction allows consumers to utilize the functionality while avoiding unnecessary dependencies on the specifics of the underlying implementation.

Additionally, inner APIs act as a contract between the teams delivering back-end functionality and the consuming developers using the API for apps, integrations and complex architectures. This contract allows the domain delivery team to operate independently from the consuming developers in terms of technology choices and delivery cadences.

Example Technologies

The following are examples of the types of API formats that may be used to define inner APIs:

- AsyncAPI
- GraphQL
- gRPC (Protocol Buffers)
- OpenAPI Specification (REST APIs)

Key Characteristics

Inner APIs have the following characteristics:

- Allow connectivity to include both event communication and API communication.

This research note is restricted to the personal use of .

- Expose the functionality of applications, systems and services, so that they are accessible by a mediation layer.
- Are tailored to the domain constraints of the underlying service to support access from multiple consumers.
- May be used to service communication within their domain.
- Are typically owned by the teams providing the underlying application, system or service functionality.
- Are designed to give consumers access to data, functionality and other resources for an internal system/service, based on the internal domain model, process workflow and native communication model.
- Decouple the service deployment from the API deployment to enable flexible, agile and scalable implementation and modification of services.
- Decouple the versioning of APIs from the versioning of the underlying service implementation, allowing the service to be maintained and updated as needed without affecting consumers.
- Abstract the underlying technical implementation details to:
  - Avoid giving consumers unwanted access to the underlying system.
  - Minimize dependencies that would prohibit updating/modernizing the underlying technology.

Related Architecture Research

How to Design Great APIs

How to Deliver Sustainable APIs

Choosing an API Format: REST Using OAS, GraphQL, gRPC or AsyncAPI

Reference Architecture Brief: Event-Driven Application Architecture

This research note is restricted to the personal use of .

Multigrained Services

Multigrained services comprise the underlying systems, services, data and other resources used by multiexperience apps, integrations and complex solutions within a heterogeneous distributed architecture. From an architecture perspective, multigrained services include three major components:

- **Service implementation:** The functionality of multigrained services can come from a wide variety of technologies. These include, for example, custom services/microservices, AI agents, AI services, backend for frontend (BFF) service, enterprise applications, API-enabled SaaS, serverless functions, data streaming, data services/virtualization and even legacy applications wrapped in an API.
- **Inner APIs:** These are API or event interfaces that enable access to underlying service implementations.
- **Application state:** This component includes two elements:
  - *The data required to persist the application state either within a specific service or across multiple multigrained services.* This state may exist in technologies such as the natural data storage or databases of the service implementation, systems of record, the storage/database of BFF services, or a memory cache.
  - *The content used to render multiexperience app interfaces.* Such content includes static content stored in a file system, dynamic content rendered in real time by a back-end service, and content from technologies, such as content management systems or digital experience platforms.

Example Technologies

The following are examples of the types of technologies that may be associated with multigrained services:

- **Custom services languages and frameworks:**
  - C# Microsoft .NET Core, Java Spring Boot, JavaScript Node.js with Express, Python Flask
- **Cloud application runtimes:**
  - Amazon Web Services (AWS) App Runner, Google Cloud App Engine, Microsoft Azure App Service

This research note is restricted to the personal use of .

- **Cloud serverless platforms:**
  - AWS Lambda, Google Cloud Run Functions, Microsoft Azure Functions
- **Service mesh technologies:**
  - AWS App Mesh, Google Anthos Service Mesh, Istio Service Mesh, Microsoft Open Service Mesh (OSM), Red Hat OpenShift, Solo.io Gloo Mesh
- **Enterprise and SaaS application vendors with APIs:**
  - IBM, Google, Microsoft, Oracle, Salesforce, SAP
- **Data virtualization and integration:**
  - Denodo Platform, Google Cloud Platform, IBM Cloud Pak, Informatica Intelligent Data Management Cloud, Microsoft Fabric

Key Characteristics

Multigrained services have the following characteristics:

- Include an API or event interface that is consumable within a mesh app and service ecosystem.
- Employ a wide variety of existing implementation technologies, ranging from enterprise application APIs to cloud services to custom-built applications and services.
- Range in granularity from monolithic applications exposing a macroservice interface to coarse-grained miniservices to fine-grained microservices.
- Rightsize service implementations to minimize the number of services while accommodating independent delivery cadence, technology choice and other constraints.
- Decouple service versioning from inner-API versioning, allowing services to be maintained and updated as needed without affecting consumers.
- Are designed to support continuous delivery and integrations.
- Clearly define and document dependencies for API consumers.

This research note is restricted to the personal use of .

- Utilize the appropriate runtime technologies and frameworks to align with business requirements.
- Are cloud-native when possible to enable latency-tolerant, instrumented, failure-tolerant, event-driven, secure in-depth, parallelizable, automated and resource-aware (LIFESPAR) principles.
- Decompose macroservices as separate services when necessary to enable independent delivery cadence, scaling and technology choice.
- May combine microservices when necessary to reduce communication and operational overhead.
- Implement a modular design that enables them to be further decomposed and refactored with minimal effect on consuming applications.
- Allow mesh application state to support both an experience-specific state and a distributed state that can comprise data embedded into the state of one or more multigrained services.

Related Architecture Research

How to Use LIFESPAR Architecture Principles to Deliver Cloud-Native Applications

Decision Point for Choosing a Runtime Platform for Cloud-Native Applications

Assessing OpenShift for Cloud-Native Application Delivery on Kubernetes

JavaScript: Top Use Cases, Frameworks and Architecture Constraints

Key Architecture Principles & Patterns

The following architecture principles and patterns apply to composable application architecture:

- **Fit-for-purpose user experiences:** User experiences are designed as narrowly focused apps that align with the specific workflows required by different personas. Rather than focusing on all features that every type of user may require in the application, fit-for-purpose apps focus on the workflows that most users follow as they perform their specialized tasks.

This research note is restricted to the personal use of .

- **Multiexperience apps:** A mesh application user experience (UX) is composed of one or more apps that provide a continuous experience for a user persona's workflows across that user's chosen device types and modality of interaction.
- **Multigrained service ecosystem:** Service architecture utilizes a heterogeneous mix of technologies and service granularity. Instead of rigid service architectures, such as monolithic, layered or even strict microservices, the architecture treats all back-end functionality as peers via access to APIs and event interfaces. This democratization allows you to more easily design composite architectures based on the functionality provided by the API, rather than on the service implementation technology.
- **Composite (or mesh) applications:** Applications comprise a mix of multiexperience apps and multigrained services. This pattern enables composable architecture, where applications are built by connecting apps and services along with functionality and data.
- **Distributed architecture:** Logic, data and functionality may be distributed across multiple technologies and geographical regions. This principle allows you to optimize the architecture more efficiently by, for example, creating location-based apps, keeping data-bound logic close to its data and utilizing hybrid cloud architecture.
- **Modular code design:** A modular code design, in both the multiexperience apps and the back-end services, decomposes functionality into discrete components. This modularity makes it easier to understand the code, maintain the code, and adapt the code for changing user and business requirements.
- **Cloud-native architecture:** Designing the custom services to be cloud-native will allow you to utilize cloud platform technologies to provide consistency, elastic scaling, self-service, automation, agility, lower cost and service availability.
- **Decoupled and abstracted components:** Each component, multiexperience app or multigrained service in the architecture is decoupled from other components by either inner or outer APIs and event interfaces. Additionally, the implementation details of the component are abstracted to avoid unnecessary dependencies between components. This decoupling and abstraction allow each component to be built, tested, deployed and operated independently of the other components.

This research note is restricted to the personal use of .

- **Distributed state management:** You can design and manage each app and service independently while simultaneously architecting the overall state of the composite application. For example, you can control overall state through clever use of caching, observability and a single source of truth to improve performance, reliability and consistency in the web app data.
- **API mediation:** API mediation enables you to design and manage communication interfaces within the architecture. This includes monitoring and enforcing runtime policies for security and traffic management. API mediation is also the conduit for basic translation of protocols, formats, data structures and other transformations as requests pass between inner APIs and outer APIs.
- **Encapsulation:** You can wrap legacy systems and services with an API to enable access to their data and functionality without direct coupling.

Key Architecture Recommendations

A composable application architecture of apps, APIs and services supports agility, flexibility, integration and modularity. The following recommendations highlight when to choose composable application architecture and how to implement it effectively:

- Use a composite architecture in new applications when you need a high degree of scalability, flexibility, custom integration and composability.
- Use this architecture as the optimal approach to modernizing legacy applications.
- Use this architecture as an effective approach to incorporate AI capabilities into, and alongside, existing applications.
- Implement composite architecture in steps, incrementally building agile components and adapting functionality in each iteration. This incremental approach allows you to evolve the architecture, hone your skills, and apply best practices as additional requirements for apps, APIs and services become clear.
- Decompose large monolithic user experiences that align with multiple roles to enable fit-for-purpose apps that align with user workflows. In other words, create apps that align with how people live their lives.
- Use a mediated API approach to decouple and abstract back-end service dependencies from multiexperience apps.

This research note is restricted to the personal use of .

- Treat APIs and event schemas as contracts that define what is expected of both the API producer and the API consumers. This approach will help you avoid dependencies that limit agility. Additionally, these contracts provide a basis for discussions, collaboration and planning as teams that own different components work together with mesh application teams to deliver value.
- Implement a versioning strategy that isolates the life cycle for each architectural component to minimize coupling and to provide the flexibility that enables you to adapt and innovate.
- Distribute data, logic and application state to optimize user experience, back-end scale/performance, use of technologies and geographic location.
- Apply proven integration patterns to enable connectivity between multiple back-end services, systems and applications.
- Balance the agility and flexibility gained via more fine-grained apps and services with the operational and architectural complexity created by going too granular.
- Ensure a mesh application team is in place to design the overall composite mesh application architecture and to manage details involved in ownership and governance of individual components.
- Ensure you have an effective API management strategy that includes how policies are defined and enforced. In addition, make sure you have an API catalog that is visible to teams creating composite architectures.
- Avoid composable architecture if:
  - Your applications are built on a single technology.
  - You do not need a distributed architecture with multiple apps, APIs and services.
  - You do not have the architectural experience to implement a distributed architecture.

Recommended by the Author

Some documents may not be available as part of your current Gartner subscription.

How to Deliver Sustainable APIs

This research note is restricted to the personal use of .

How to Design Great APIs

MASA: Create Agile Application Architecture With Composable Apps, APIs and Services

Reference Architecture Brief: Agile Web Application Architecture

Using MASA to Deliver Agile Enterprise Application Architecture

Reference Architecture Brief: API Management

Reference Architecture Brief: Microservices Architecture Delivery Platform

Reference Architecture Brief: Event-Driven Application Architecture

---

© 2025 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's research organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner research may address legal and financial issues, Gartner does not provide legal or investment advice and its research should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "Guiding Principles on Independence and Objectivity." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .